A GPU-Accelerated DREAMPlace/OpenROAD Evaluator Node for CHIA

Timing-Driven GPU Placement, Cached Multi-Backend Evaluation, and Noise-Aware Agentic Search

Kaviprashanna Krishnamoorthy
Independent Researcher
India

Abstract

Agents that tune physical-design flows need an evaluator that is fast, accurate and reusable. A full place-and-route run of a large design takes a long time, and its quality varies with the placement seed. We present a reusable CHIA evaluator node built for this. One request/result contract drives two flow backends: OpenROAD-flow-scripts (ORFS) on NanGate45 and LibreLane on sky130. Placement runs on the GPU and routing on the CPU as separate CHIA functions. Every result is cached by the content of its request, so replaying a finished experiment runs no tools. The node places with DREAMPlace, made timing-driven by reweighting critical nets from OpenSTA slack. On Ariane133 it beats stock ORFS on timing (TNS −323 vs. −530 ns, WNS −0.226 vs. −0.322 ns), with 1.8× faster placement and comparable end-to-end runtime. On three smaller designs TNS ties and routed wirelength is 2–5% shorter. On top of the node, five proposers, two of them LLMs, tune eight flow knobs. A routed final round keeps the default unless a candidate beats it by more than seed noise.

Keywords

physical design, placement, OpenROAD, DREAMPlace, LLM agents, CHIA, design-space exploration

ACM Reference Format:
Kaviprashanna Krishnamoorthy. 2026. A GPU-Accelerated DREAMPlace/OpenROAD Evaluator Node for CHIA: Timing-Driven GPU Placement, Cached Multi-Backend Evaluation, and Noise-Aware Agentic Search. In *A^3: Agentic Approaches to Architecture Workshop at MICRO 2026; Hackathon Short Paper, Nov 01, 2026, Athens, Greece.* ACM, New York, NY, USA, 4 pages.

1 Introduction

Physical-design flows expose many parameters that trade off power, performance and area (PPA), and recent work automates their tuning, from black-box search to LLM agents. AutoTuner [6] searches ORFS parameters with derivative-free optimization, AutoDMP [1] tunes DREAMPlace macro placement with Bayesian optimization, and ChatEDA [10] uses an LLM agent to plan and run RTL-to-GDS flows. Such loops are slow and noisy. A place-and-route run of a large design takes a long time, its quality changes with the placement seed, and a cheap estimate such as the placer's own objective can rank candidates in the wrong order. An agent that tunes the flow therefore needs an evaluator that is fast, returns accurate results, and can be reused across designs, flows and proposers. CHIA [5] provides the agent loop; we contribute such an evaluator.

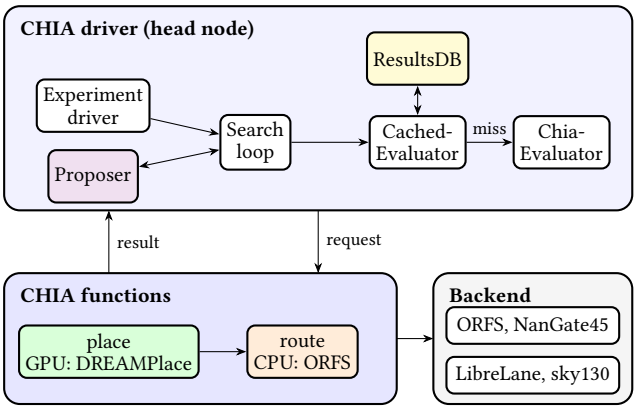


Figure 1: The evaluator node in CHIA. Driver, CHIA functions and flow backends.

DREAMPlace [8] treats analytical global placement as neural-network training. The cell positions are the trainable parameters. The loss is a smooth wirelength model, whose smoothness is set by a parameter γ, plus a density penalty that pushes local cell density toward a target density. A gradient optimizer updates all positions at once with GPU kernels. Placement stops when the overflow, the fraction of cell area above the target density, falls below a threshold. OpenROAD [2] and its flow scripts (ORFS [9]) form an open RTL-to-GDS flow that places with RePlAce [4], and LibreLane [7] adds Magic/Netgen signoff on sky130. Our contributions are:

- A reusable CHIA evaluator node: one request/result contract over two flow backends (ORFS on NanGate45, LibreLane on sky130), placement and routing as separate GPU and CPU CHIA functions, and content-addressed caching that replays finished experiments without rerunning tools (§2).
- Timing-driven GPU placement inside ORFS: DREAMPlace interleaved with OpenROAD buffer repair and OpenSTA net weights. It beats stock ORFS on Ariane133 timing with placement 1.8× faster (§2, §4).
- A noise-aware search protocol (§3). Candidates are scored after clock-tree synthesis, the cheapest stage whose timing predicts the routed ranking. A routed final round accepts a candidate only if it beats the default by more than seed noise.

This work is licensed under a Creative Commons Attribution 4.0 International License.
A^3 Workshop at MICRO 2026; Hackathon Short Paper, Athens, Greece
© 2026 Copyright held by the owner/author(s).

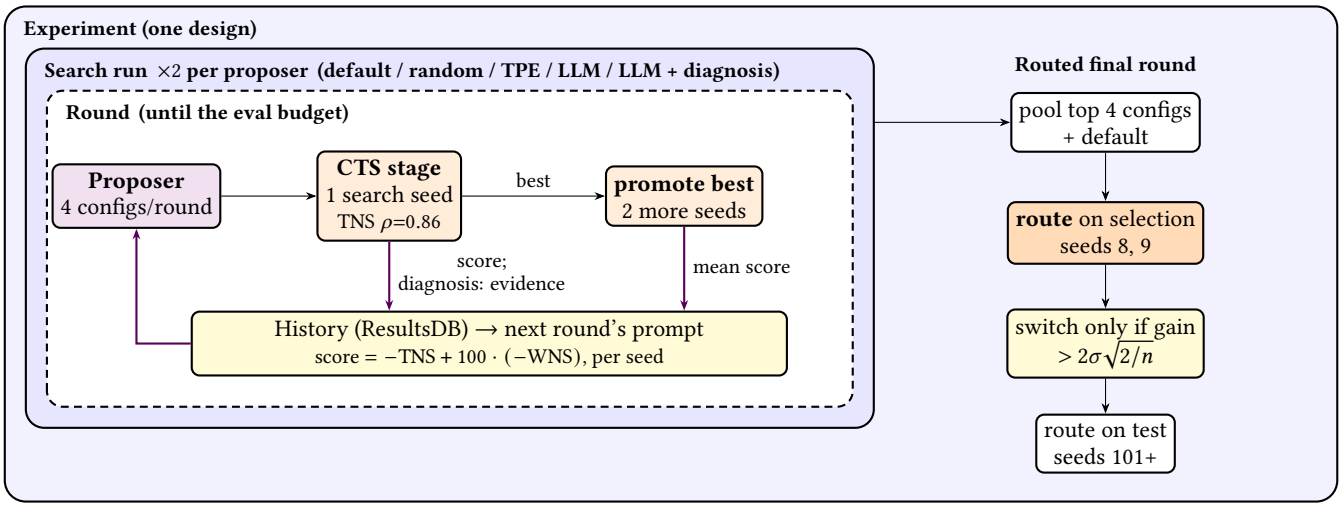


Figure 2: Evaluation protocol. Settings as used on Ariane133.

- A comparison of five proposers (default, random, a tree-structured Parzen estimator (TPE) [3], an LLM and an LLM with timing diagnosis) on four designs (§4). On jpeg and tinyRocket the final round keeps the default. On swerv_wrapper it adopts an LLM-proposed configuration that improves worst negative slack (WNS) on 11 of 12 test seeds. Its score gain is within seed noise.

2 The Evaluator Node

Contract and backends. The node sits between a CHIA search and the flow tools (Fig. 1). On the driver side, the experiment driver and the search loop turn each proposer's configurations into requests. A `CachedEvaluator` answers requests it has already seen and sends the rest to CHIA functions on the workers. Placement runs as a GPU CHIA function and routing as a CPU-only one, so routing never holds a GPU. An `EvalRequest` names a design, technology, knob settings, seed, stage and toolchain fingerprint. An `EvalResult` carries wirelength (WL), WNS and total negative slack (TNS) per stage, power, area, the detailed-route design-rule-check (DRC) count and provenance. The driver, search loop and proposers see only these two records, so they run unchanged on either flow backend, chosen per experiment: ORFS on NanGate45 or LibreLane on sky130. All experiments in this paper use the ORFS backend. The LibreLane backend uses the same DREAMPlace handoff and reaches Magic DRC = 0 and Netgen layout-versus-schematic (LVS) = 0 on `spm`.

GPU placement handoff. Stock (unmodified) ORFS runs until IO placement and writes the design as a DEF file. DREAMPlace places it on the GPU, and we read the cells back into OpenROAD. ORFS then continues unmodified through resizing, legalization, clock-tree synthesis (CTS), global and detailed routing and final reports. The final placement is DREAMPlace's, snapped to legal sites. As a baseline, AutoDMP's best tinyRocket trial, run through a format conversion outside its tested setup, routed with 159 DRC violations.

Timing-driven GPU placement. Stock ORFS's global placer pauses twice, at overflow 0.63 and 0.20, to repair the netlist and reweight critical nets. We do the same around DREAMPlace. DREAMPlace runs until the overflow falls to 0.6. OpenROAD then loads the placement, runs `repair_design` and then re-buffers every net, and computes per-net slack with OpenSTA. The worst 10% of nets get weights of up to 5 for the next DREAMPlace run, which resumes from the repaired placement. This repeats at overflow 0.2, and a last run finishes at 0.1. The repaired netlist moves between stages as an OpenROAD database, so later stages start from the repaired netlist instead of redoing the repairs. Before the first repair, nets with more than 2000 loads are split by a 5×5 grid of buffers, so that `repair_design` does not spend minutes on a single net. On Ariane133 this splits the 20,636-load reset net with 19 buffers and cuts `repair_design` from 480 to 84 s. Target density is 0.45 (set on Ariane133 and used for all designs). We call the result the *GPU flow*.

Determinism and caching. On gcd, repeating a request gives bit-identical results, even with 2 or 8 threads. A request's identity is a hash of its fields, including the image digest and DREAMPlace commit. A `CachedEvaluator` replays known identities. Replaying two finished experiment databases resolved all 804 (gcd) and 201 (tinyRocket) identities without a single new evaluation. The searches in this paper served 5, 2 and 6 results from cache on Ariane133, jpeg and tinyRocket.

3 Evaluation Protocol

Search space and proposers. The protocol (Fig. 2) has two stages: a search scored after CTS, then a routed final round. Each proposer runs the search more than once, and each repeat is a *search run*. The search space is eight flow knobs: target density, density weight, wirelength smoothing γ, learning rate, the overflow values at which DREAMPlace pauses for repair, full or incremental re-buffering, and the share of nets that get weights and the maximum weight. The objective puts timing first: score $= -\text{TNS} + 100 \cdot (-\text{WNS})$, in ns. Lower is better, and 10 ps of WNS counts as much as 1 ns of TNS. The random proposer samples uniformly (log-uniformly for density weight and learning rate). The LLM proposer (Gemini 3.1 Pro

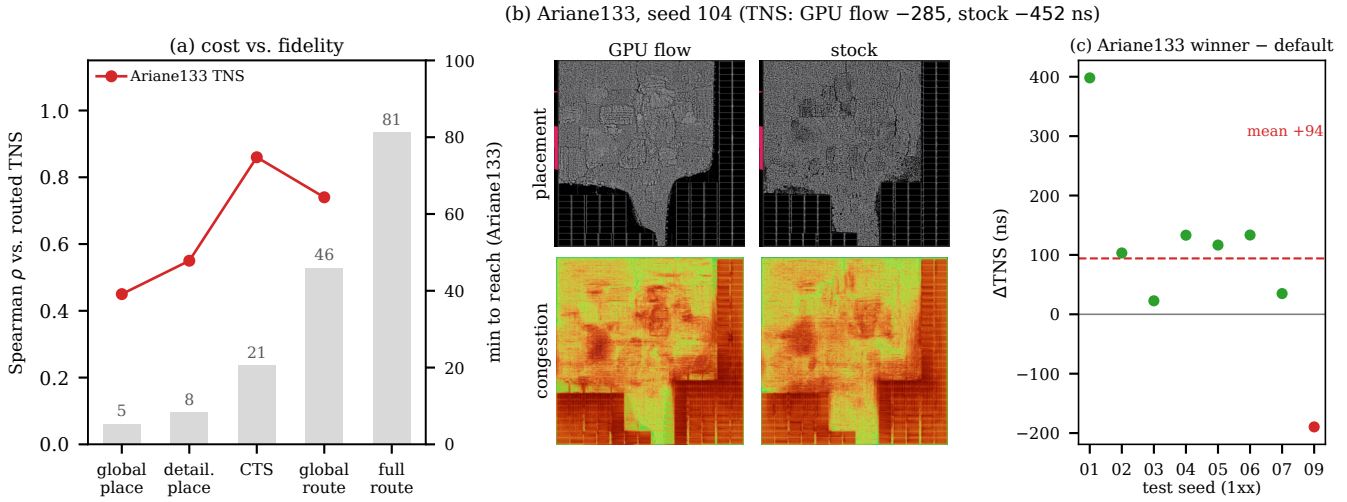


Figure 3: Ariane133 results. (a) Rank correlation with the routed result per stage, over the wall time to reach each stage. (b) Placement (top) and routing congestion (bottom), seed 104. (c) TNS change of the Ariane133 winner over the default, per test seed (positive = better).

preview) makes one call per round, which returns configurations as JSON, with the knob table and its scored history. The diagnosis proposer also sees what each run showed (per-stage repair and congestion data, the worst paths and high-fanout nets) and must state a hypothesis for each configuration. On Ariane133 each search run gets two rounds of four configurations on search seed 3, and each round's best is re-run on seeds 4 and 5.

Cost and fidelity. Routing every candidate is too slow, so search stops at an earlier stage. That stage's numbers need not match the routed result; they only have to put candidates in the same order. Fig. 3a compares the stages on 13 competitive Ariane133 runs (final TNS better than −700 ns). For each stage it shows how well its TNS ranks the routed TNS, and how many minutes it takes to reach. After placement the ranking is weak (Spearman 0.45, after 5 minutes). After CTS it is strong (0.86, after 21 minutes), and global route is no better (0.74, after 46). Search stops at CTS, about a quarter of the 81 minutes of a full route. The placer's own estimate is not a safe stage for wirelength either: on 60 gcd configurations, its half-perimeter wirelength ranks the routed wirelength with Spearman −0.29.

Noise and the routed final round. The flow is deterministic for a fixed seed, but routing amplifies small placement differences between seeds. The protocol uses three disjoint sets of seeds: search seeds (3–7 on Ariane133), selection seeds (8, 9) and test seeds (101 and up). Test seeds are never used by the search or the final round. We define σ as the standard deviation of the default configuration's score over five search seeds. On Ariane133, σ is 17% of the mean score. A proposer's result counts only if, on the test seeds, it beats the default by more than twice the standard deviation of the default's routed score across those seeds. CTS is only a filter: on swerv_wrapper, a configuration three times better than the default at CTS routed worse. After search, the four distinct configurations with the best mean CTS score over at least two search seeds become the *finalists*. In practice these are configurations promoted as the best of their round. The finalists and the default are routed on the two selection seeds. A finalist replaces the default only if it wins by more than a threshold $\theta = 2\sigma\sqrt{2/n}$, with $n = 2$ selection seeds. The winner is then compared with the default on the test seeds.

Table 1: GPU flow vs. stock ORFS on test seeds. Δ = GPU − stock, ±2 standard errors (SE) of independent groups; positive ΔTNS and ΔWNS favour the GPU flow; negative ΔWL means shorter wire. All routes: 0 DRC.

design (cells/macros)	runs	TNS GPU/stock	ΔTNS	ΔWNS	ΔWL
Ariane133 (161k/132)	4	−323/−530	+208±169	+0.096±0.068	+2.9%
swerv_wrapper* (95k/28)	8	−69/−62	−7±25	−0.059±0.015	−5.3%
jpeg (60k/0)	4	−34/−38	+3.8±5.3	+0.008±0.012	−2.2%
tinyRocket (29k/2)	8	−30/−33	+2.9±9.0	+0.002±0.008	−3.3%

*Flat netlist in both flows (§5).

Table 2: Routed final round. Lead = default's score − best finalist's score on the selection seeds (higher is better). Test Δscore = default − winner (higher is better), paired by seed, ±2 SE.

design	lead	θ	chosen	test seeds	test Δscore
Ariane133	324.5	121.9	finalist	8	+99±120
swerv_wrapper	113.2	55.0	finalist	12	+20±26
jpeg	1.6	11.8	default	4	+0.3±5.7†
tinyRocket	5.0	11.9	default	4	−4.8±4.6†

†Best finalist, not adopted: its test Δscore had the round adopted it.

4 Results

Against stock ORFS. On Ariane133, the design with the most cells and macros, the GPU flow beats stock on both TNS and WNS (Table 1), at 2.9% longer wirelength. Fig. 3b shows one seed of each. On the three smaller designs TNS ties stock and wirelength is 2–5% shorter. WNS on swerv_wrapper is worse (−0.183 vs. −0.124 ns).

Search and final round. Table 2 shows its results. The round picks an LLM-proposed configuration for swerv_wrapper that ranked last of four at CTS. Over 12 test seeds it improves WNS by 0.033 ± 0.013 ns, on 11 of them, and shortens wirelength by 1.7% ± 0.4%, on all 12. Its score change in Table 2 is within noise, because the score is dominated by TNS, which varies widely between seeds on this design. On jpeg and tinyRocket the best finalist's lead is within the threshold, so the default is kept. Adopting those finalists would have tied the default on jpeg and lost to it on tinyRocket (Table 2, †). On Ariane133 the best finalist's lead clears the threshold, which gives the *Ariane133 winner*.

On jpeg and tinyRocket every proposer's search-stage lead (up to 25%) vanished on test seeds. On the two macro-heavy designs the LLM proposers found the configurations the final round picked. The Ariane133 winner, found by the diagnosis proposer, changes three knobs: it pauses for repair at overflow 0.6, 0.4 and 0.2 instead of 0.6 and 0.2, raises the maximum net weight to 8 and sets $\gamma = 2$. The Ariane133 winner improves TNS on 7 of 8 test seeds, by +94 ± 115 ns on average. Its per-seed TNS change ranges from −190 ns to +398 ns (Fig. 3c). With two or three search runs per proposer, no proposer's lead clears this bound, so the comparison does not rank the proposers.

Runtime. Both flows were timed one run at a time, on the same machine under the same load. On Ariane133, placement takes 498 s with the GPU flow vs. 887 s for stock, 1.8× faster. The GPU placement itself takes about 10 s, and the rest is OpenROAD repair and timing analysis. End to end the flows are on par: the difference is within the run-to-run spread. On tinyRocket the GPU flow is 1.4× slower.

5 Limitations

Only global placement runs on the GPU. Timing-driven placement still needs OpenROAD repair and OpenSTA on the CPU, and these take nearly all of the placement time. CTS and routing take most of the end-to-end runtime. As a result, the GPU flow matches stock end to end and is slower on small designs. The evidence covers four designs in one technology, NanGate45. Only one of them, Ariane133, is large and macro-dense, and that is where the GPU flow improves timing. Our per-net slack step requires a flat netlist, so hierarchical designs are flattened, and stock ORFS still has better WNS on swerv_wrapper. With two or three search runs per proposer and 4–12 test seeds per design, the study cannot rank the proposers. CTS was validated as the search stage on Ariane133 only. The LibreLane backend was tested on individual designs and not used for search.

6 Conclusion

We presented a reusable CHIA evaluator node with one request/result contract over two flow backends, placement and routing as separate GPU and CPU CHIA functions, and content-addressed caching. Its timing-driven GPU placement beats stock ORFS on Ariane133 timing with 1.8× faster placement. On three smaller designs it ties stock on TNS with shorter wirelength. Its noise-aware protocol keeps the default when search gains are within seed noise. On Ariane133 the adopted configuration's gain is within noise on test seeds; on swerv_wrapper the routed round adopts an LLM-proposed configuration that ranked last at CTS and improves WNS on 11 of 12 test seeds. Its score gain is within seed noise.

7 Artifact Information

The artifact is at https://github.com/kavsdev/gpu-pnr-chia/tree/a3-camera-ready, the release that matches this paper. It contains the `pnr_node` package (contract, CHIA functions, DREAMPlace handoff, timing-driven placement, cache, search loop, proposers and final round) and a unit-test suite that needs no EDA tools or GPU. It also contains the Docker environment, the job file for every run, and every cited result with its source report and raw data. The toolchain is the ORFS image `openroad/orfs@sha256:573c1716. . .` and DREAMPlace commit `6627f33` with the patches in `docker/`. One script (`docker/run_local.sh`) builds it.

Use of AI Assistance

AI assistance was used in the development and writing of this work.

References

[1] Anthony Agnesina, Puranjay Rajvanshi, Tian Yang, Geraldo Pradipta, Austin Jiao, Ben Keller, Brucek Khailany, and Haoxing Ren. 2023. AutoDMP: Automated DREAMPlace-based Macro Placement. In *Proceedings of the 2023 International Symposium on Physical Design (ISPD)*. ACM, New York, NY, USA, 149–157. doi:10.1145/3569052.3578923

[2] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Lutong Wang, Zhehong Wang, Mingyu Woo, and Bangqi Xu. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*. ACM, New York, NY, USA, 1–4. doi:10.1145/3316781.3326334

[3] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for Hyper-Parameter Optimization. In *Advances in Neural Information Processing Systems 24 (NIPS)*. Curran Associates, Inc., Red Hook, NY, USA, 2546–2554.

[4] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. 2019. RePlAce: Advancing Solution Quality and Routability Validation in Global Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 9 (2019), 1717–1730. doi:10.1109/TCAD.2018.2859220

[5] Angela Cui, Ferran Hermida-Rivera, Jack Toubes, Raghav Gupta, Jim Fang, Chengyi Lux Zhang, Ella Schwarz, Junha Kim, Yakun Sophia Shao, Borivoje Nikolic, Christopher W. Fletcher, and Sagar Karandikar. 2026. CHIA: An Open-Source Framework for Principled, Agentic AI-Driven Hardware/Software Co-Design Research. arXiv:2606.27350 [cs.AR]. https://arxiv.org/abs/2606.27350

[6] Jinwook Jung, Andrew B. Kahng, Seungwon Kim, and Ravi Varadarajan. 2021. METRICS2.1 and Flow Tuning in the IEEE CEDA Robust Design Flow and OpenROAD. In *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

[7] LibreLane contributors. 2026. LibreLane. https://github.com/librelane/librelane.

[8] Yibo Lin, Shounak Dhar, Wuxi Li, Haoxing Ren, Brucek Khailany, and David Z. Pan. 2019. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. In *Proceedings of the 56th Annual Design Automation Conference (DAC)*. ACM, New York, NY, USA, 1–6. doi:10.1145/3316781.3317803

[9] The OpenROAD Project. 2026. OpenROAD-flow-scripts. https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.

[10] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3184–3197. doi:10.1109/TCAD.2024.3383347